

Agile Modeling, Prototyping, and Scrum

LEARNING OBJECTIVES

Once you have mastered the material in this chapter, you will be able to:

1. Understand the roots of agile modeling in prototyping and the four main types of prototyping.
2. Understand agile modeling and the core practices that differentiate it from other development methodologies.
3. Understand Scrum as an agile method to improve development of complex projects.
4. Learn about DevOps as a cultural shift in the way to organize rapid systems development and operations.
5. Learn the importance of values critical to agile modeling.
6. Understand how to improve efficiency for users who are knowledge workers using either structured methods or agile modeling.

This chapter explores agile modeling, which is a collection of innovative, user-centered approaches to systems development. You will learn the values and principles, activities, resources, practices, processes, and tools associated with agile methodologies. Agile approaches have their roots in prototyping, so this chapter begins with prototyping to provide a proper context for understanding, and then it takes up the agile approach later in the chapter.

Agile modeling is distinct from structured methods as a systems development approach that helps develop systems rapidly. Scrum is an agile method that is suitable for more complex systems projects. Using Scrum, a limited number of features or tasks are selected for completion in a sprint, typically lasting two to four weeks. The result of the sprint is a potentially shippable product. Once the sprint ends, the process begins again with new priorities and features to be finished in the next sprint. There are specific roles to play in Scrum, and there are also specific actions to take.

DevOps (derived from Development and Operations) is a cultural shift in the organization. It can create two approaches in the organization that run on parallel tracks, one track supports developing rapid innovative applications, and the other supports the maintenance and operations of those processes already in place.

Prototyping

A systems analyst presenting a prototype of an information system is keenly interested in the reactions of users and management to the prototype. The analyst wants to know in detail how they react to working with the prototype and how good the fit is between their needs and the prototyped features of the system. Reactions are gathered through observation, interviews, and feedback sheets (possibly questionnaires) designed to elicit each person's opinion about the prototype as he or she interacts with it.

Information gathered in the prototyping phase allows an analyst to set priorities and redirect plans inexpensively, with minimal disruption. Therefore, prototyping and planning go hand in hand.

Kinds of Prototypes

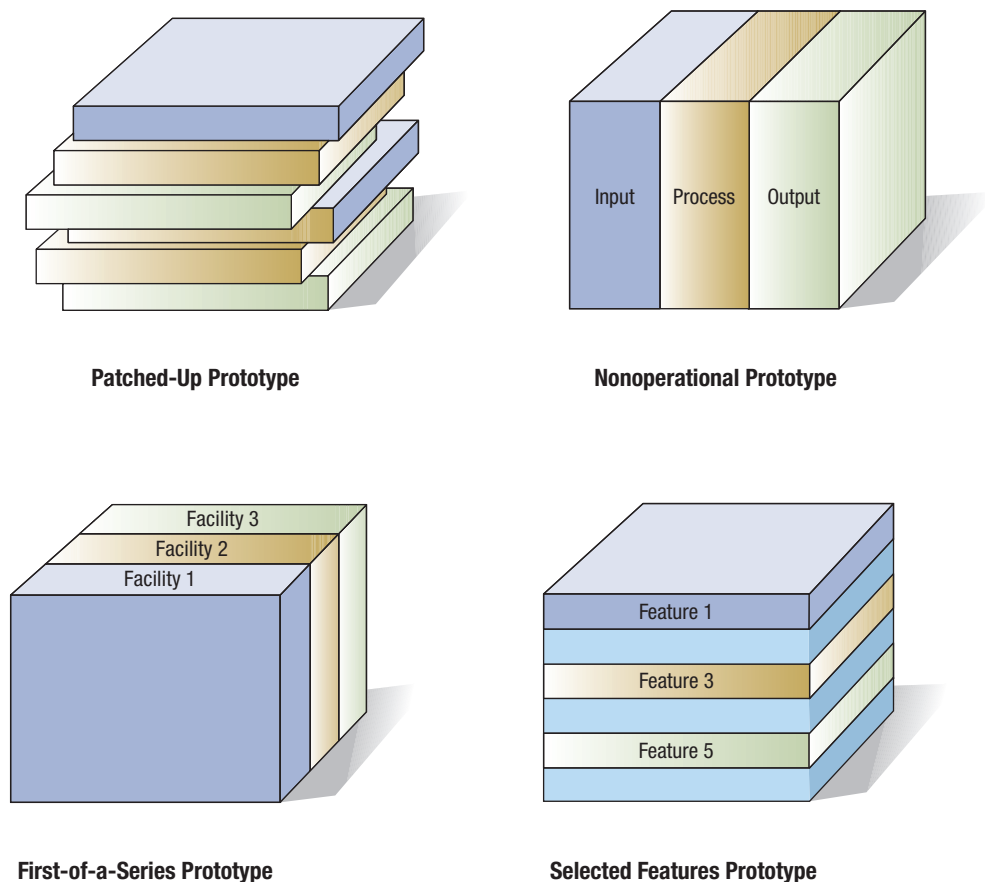
The word *prototype* is used in many different ways. Rather than attempt to synthesize all these uses into one definition or try to mandate one correct approach to the somewhat controversial topic of prototyping, we illustrate how each of several conceptions of prototyping may be usefully applied in a particular situation, as shown in Figure 6.1.

PATCHED-UP PROTOTYPE The first kind of prototyping has to do with constructing a system that works but is patched up or patched together. In engineering, this approach is referred to as breadboarding: creating a patched-together, working model of an (otherwise microscopic) integrated circuit.

An example in information systems is a working model that has all the necessary features but is inefficient. In this instance of prototyping, users can interact with the system and get accustomed to the interface and the types of output available. The retrieval and storage of information may be inefficient, however, because applications were written rapidly, with the objective of being workable rather than efficient.

FIGURE 6.1

Four kinds of prototypes.





CONSULTING OPPORTUNITY 6.1

Is Prototyping King?

“As you know, we’re an enthusiastic group. We’re not a dynasty yet, but we’re working on it,” Paul LeGon tells you. At 24 years of age, Paul (introduced in Consulting Opportunity 2.3) is the “boy king” of Pyramid, Inc., a small but successful independent publishing firm that specializes in paperback books outside of the publishing mainstream. As a systems analyst, you have been hired by Pyramid, Inc. to help develop a computerized warehouse inventory and distribution information system.

“We’re hiring lots of workers,” Paul continues, as if to convince you of the vastness of Pyramid’s undertaking. “And we feel Pyramid is positioned perfectly as far as our markets in the north, south, east, and west are concerned.”

“My assistant, Ceil Toom, and I have been slaving away, thinking about the new system. And we’ve concluded that what we really need is a prototype. As a matter of fact, we’ve tunneled through a lot of material. Our fascination with the whole idea has really pyramided.”

As you formulate a response to Paul, you think back over the few weeks you’ve worked with Pyramid, Inc. You think that the business problems its information system must resolve are very straightforward. You also know that the people in the company are on a limited budget and cannot afford to spend like kings. Actually, the entire project is quite small.

Ceil, building on what Paul has said, tells you, “We don’t mean to be too wrapped up with it, but we feel prototyping represents the new world. And that’s where we all want to be. We know we need a prototype. Have we convinced you?”

Based on Paul’s and Ceil’s enthusiasm for prototyping and what you know about Pyramid’s needs, would you support construction of a prototype? Why or why not? Formulate your decision and response in a letter to Paul LeGon and Ceil Toom. Present a justification for your decision based on overall criteria that should be met to justify prototyping.

NONOPERATIONAL PROTOTYPE The second conception of a prototype is that of a nonworking scale model that is set up to test certain aspects of the design. An example of this approach is a full-scale model of an automobile that is used in wind tunnel tests. The size and shape of the auto are precise, but the car is not operational. In this case, only features of the automobile that are essential to wind tunnel testing are included.

A nonworking scale model of an information system might be produced when the coding required by the applications is too extensive to prototype but when a useful idea of the system can be gained through the prototyping of the input and output only. In this instance, processing, because of undue cost and time, would not be prototyped. Users could still make decisions on the utility of the system, based on their use of prototyped input and output.

FIRST-OF-A-SERIES PROTOTYPE A third conception of prototyping involves creating a first full-scale model of a system, often called a pilot. An example is prototyping the first airplane of a series and then seeing whether it flies before building a second. The prototype is completely operational and is a realization of what the designer hopes will be a series of airplanes with identical features.

This type of prototyping is useful when many installations of the same information system are planned. The full-scale working model allows users to experience realistic interaction with the new system, but it minimizes the cost of overcoming any problems it presents. For example, when a retail grocery chain intends to use blockchain (see Chapter 13) to check in suppliers’ shipments in a number of outlets, a full-scale implementation might be done in one store so users could work through any problems before blockchain is implemented as part of the supply chain in all the others.

SELECTED FEATURES PROTOTYPE A fourth conception of prototyping concerns building an operational model that includes some, but not all, of the features that the final system will have. An analogy would be a new retail shopping mall that opens before the construction of all shops is complete.

When prototyping information systems in this way, some, but not all, essential features are included. For example, users may view a system menu on a screen that lists six features: add a record, update a record, delete a record, search a record for a key word, list a record, or scan a

record. In the prototyped system, however, only three of the six may be available for use, so that the user may add a record (feature 1), delete a record (feature 3), and list a record (feature 5). User feedback can help analysts understand what is working and what isn't. It also can help with suggestions on what features to add next.

When this kind of prototyping is done, the system is accomplished in modules so prototyped features evaluated by users as successful can be incorporated into the larger, final system without undertaking immense work in interfacing. Prototypes done in this manner are part of the actual system. They are *not* just mock-ups, as in nonoperational prototyping. Unless otherwise mentioned, all further references to prototyping in this chapter refer to selected features prototypes.

The Users' Role in Prototyping

The users' role in prototyping can be summed up in two words: honest involvement. Without user involvement, there is little reason to prototype. The precise behaviors necessary for interacting with a prototype can vary, but it is clear that the user is pivotal to the prototyping process. Realizing the importance of the user to the success of the process, the members of a systems analysis team must encourage and welcome input and guard against their own natural resistance to changing the prototype.

To facilitate the prototyping process, the analyst must clearly communicate the purposes of prototyping to users, along with the idea that prototyping is valuable only when users are meaningfully involved. Some of the variables you should observe include user reactions to the prototype, user suggestions for changing or expanding the prototype, any innovations (such as suggestions for using the system in completely new ways), and any revision plans that might arise from working with the users. Although users will be asked to articulate suggestions and innovations for the prototype, in the end it is the analyst's responsibility to weigh this feedback and translate it into workable changes where necessary.

Agile Modeling

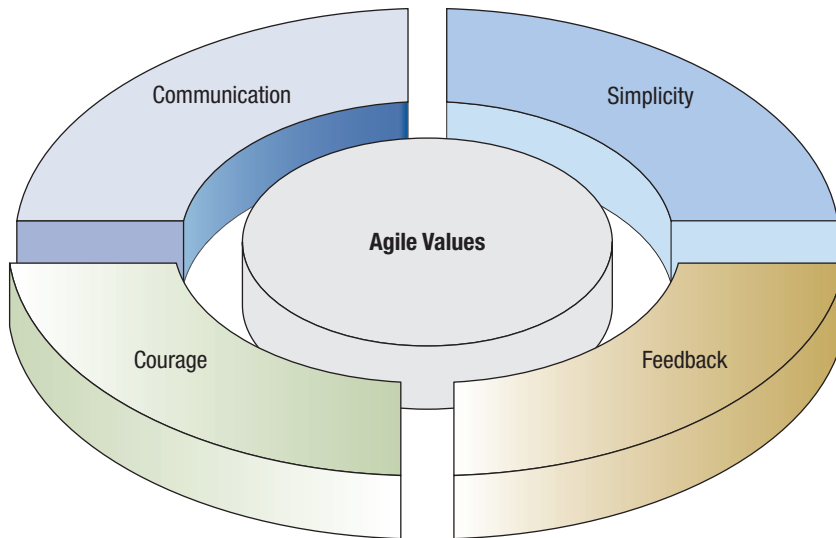
Agile methods are a collection of innovative, user-centered approaches to systems development. You will learn the values and principles, activities, resources, practices, processes, and tools associated with agile methodologies in the upcoming sections. Agile methods have been credited with many successful systems development projects and, in numerous cases, have rescued companies from a failing system designed using a structured methodology.

Values and Principles of Agile Modeling

The agile approach is not based just on results. It is based on values, principles, and practices. Essential to agile programming are stated values and principles that create the context for collaboration among programmers and customers. Agile analysts must adhere to the following values and principles as developed by Beck (2000) in his work on agile modeling, which he called "extreme programming," or "XP."

FOUR VALUES OF AGILE MODELING Four values create an environment in which both developers and businesses can adequately be served. Because there is often tension between what developers do in the short term and what is commercially desirable in the long term, it is important that analysts knowingly espouse values that will form a basis for acting together on a software project. The four values are communication, simplicity, feedback, and courage, as shown in Figure 6.2.

Let's begin with communication. Every human endeavor is fraught with possibilities for miscommunication. Systems projects that require constant updating and technical design are especially prone to such errors. Add to this tight project deadlines, specialized jargon, and the stereotype that coders would prefer to talk to machines rather than people, and you have the potential for some serious communication problems. Projects can be delayed, the wrong problem can be solved, coders may be punished for even bringing up problems to managers, people may leave or join the project in midstream without proper updates, and so the litany goes.

**FIGURE 6.2**

Values are crucial to the agile approach.

Typical agile practices such as pair programming (where two coders collaborate, described later in the chapter), tasks estimation, and unit testing rely heavily on good communication. Problems are fixed rapidly, holes are closed, and weak thinking is quickly strengthened through interaction with others on the team.

A second value of the agile approach is simplicity. When we are working on a software development project, our first inclination is to become overwhelmed with the complexity and bigness of the task. However, you cannot run until you know how to walk, nor walk until you know how to stand. Simplicity for software development means beginning with the simplest possible thing we can do.

The agile value simplicity asks us to do the simplest thing today, with the understanding that it might have to be changed a little tomorrow. This requires a clear focus on the goals of the project and really is a basic value.

Feedback is the third basic value that is important when taking an extreme programming approach. When you think of feedback in this context, it is good to consider that feedback is wrapped up with the concept of time. Good, concrete feedback that is useful to the programmer, analyst, and customer can occur within seconds, minutes, days, weeks, or months, depending on what is needed, who is communicating, and what will be done with the feedback. A fellow coder may hand you a test case that breaks the code you wrote only hours before, but that feedback is almost priceless in terms of being able to change what is not working before it is accepted and further embedded in the system.

Feedback occurs when customers create functional tests for all the stories that coders have subsequently implemented. (See more on user stories later in this chapter.) Critical feedback about the schedule comes from customers who compare the goal of the plan to the progress that has been made. Feedback helps coders make adjustments and lets the business start experiencing very early on what the new system will be like once it is fully functional.

Courage is the fourth value in agile programming. Courage has to do with the level of trust and comfort that must exist in a development team. It means not being afraid to throw out an afternoon or a day of coding and begin again if all is not right. It means being able to stay in touch with one's instincts (and test results) concerning what is working and what is not.

Courage also means responding to concrete feedback, acting on your teammates' hunches when they believe that they have a simpler, better way to accomplish a goal. Courage is a high-risk, high-reward value that encourages experimentation that can take the team to its goal more rapidly, in an innovative way. Courage means that you and your teammates trust each other and your customers enough to act in ways that will continuously improve what is being done on the project, even if that requires throwing out code, rethinking solutions, or further simplifying approaches. Courage also implies that you, as a systems analyst, eagerly apply the practices of the agile approach.

Analysts can best reflect all four values through an attitude of humility. Historically, computer software was developed by experts who often thought they knew how to run a business better than the local customers who were the true domain experts. Computer experts were often



CONSULTING OPPORTUNITY 6.2

Clearing the Way for Customer Links

World's Trend (see Chapter 7 for a detailed corporate description) is building a website solely to sell clearance merchandise usually sold through the company's main website and through its catalog operation. As a newly hired Web consultant, Lincoln Cerf finds himself in a very cold, wintry city, fighting his way through several inches of snow to meet with one of the systems team members, Mary Maye, at World's Trend headquarters.

Mary welcomes Lincoln, saying, "At least the weather doesn't seem to affect our Web sales! They're brisk no matter what." Lincoln groans appreciatively at her weak attempt at humor, smiles, and says, "I gathered from your email last week that you are trying to determine the type of information that needs to be displayed on your clearance website."

Mary replies, "Yes, I'm trying to get it organized in the best possible way. Our customers are all so busy. I know photos of all our merchandise can take a long time to load if a customer is accessing the Web via their mobile phone." Mary continues, saying, "Linc, I'm not even that concerned about how to design our clearance site at this time. I am worried, though, about how much information we need to include on a page. For example, when items are on clearance, not all colors and sizes are available. Which do you think is better, to include some basic

information and let the customer press on a tab to access more information, or to be as complete as possible on one page? If I use the linking method, then I could fit more items on the screen . . . but it might be too orderly. I do know that customers like the look and feel of a sale in which merchandise is kind of jumbled together."

Linc continues Mary's line of thought, saying, "Yeah, I wonder how customers want the information organized. Have you actually observed them using the Web? I mean, do they look for shoes when they buy a suit? If so, should shoes appear on the suit page or be linked in some way?"

Mary comments, "Those are my questions too. Then I wonder if we should just try this approach for men's clothes first, before we implement it for women's clothing. What if men's and women's approaches to shopping online are different?"

As a third member of the World's Trend website development group, respond in a brief written report to Lincoln and Mary about whether you should use a prototype to elicit recommendations from potential customers about the proposed clearance website. What type of prototype is appropriate? Consider each form of prototype and explain why each type would apply (or would not apply) to this problem. Devote a paragraph to each explanation.

referred to as "gurus." Some of the gurus displayed large egos and insisted on their infallibility, even when customers did not believe it. Many gurus lacked the virtue of humility.

However, maintaining a humble attitude during systems development is critical. You must continually embrace the idea that if the user is expressing a difficulty, then that difficulty must be addressed. It cannot be ignored. Agile modelers are systems analysts who make suggestions, voice opinions, but never insist that they are right 100 percent of the time. Agile modelers possess the self-confidence to allow their customers to question, critique, and sometimes complain about the system under development. Analysts learn from their customers, who have been in business a long time.

THE BASIC PRINCIPLES OF AGILE MODELING In a perfect world, customers and a software development team would see eye to eye, and communication would not be necessary. We would all be in agreement at all times. But we know that the ideal world doesn't exist. So how can we bring our software development projects closer to the ideal? Part of why this will not happen is that so far we are trying to operate on a vague system of shared values. They're a good beginning, but they are really not operationalized to the point at which we can measure our success in any meaningful way. So we work to derive the basic principles that can help us check whether what we are doing in our software project is actually measuring up to the values that we share.

Agile principles are the reflections and specifications of agile values. They serve as guidelines for developers to follow when developing systems. They also serve to set agile methodologies apart from the more traditional plan-driven methodologies such as the systems development life cycle (SDLC) or object-oriented methodologies.

Agile principles were first described by Beck (2000) and have evolved ever since. These principles can be expressed in a series of sayings:

1. Satisfy the customer through delivery of working software.
2. Embrace change, even if introduced late in development.

3. Continue to deliver functioning software incrementally and frequently.
4. Encourage customers and analysts to work together daily.
5. Trust motivated individuals to get the job done.
6. Promote face-to-face conversation.
7. Concentrate on getting software to work.
8. Encourage continuous, regular, and sustainable development.
9. Adopt agility with attention to mindful design.
10. Support self-organizing teams.
11. Provide rapid feedback.
12. Encourage quality.
13. Review and adjust behavior occasionally.
14. Adopt simplicity.

Often you will hear agile developers communicate their point through sayings like these or even simpler phrases, such as “model with a purpose,” “software is your primary goal,” and “travel light,” a way of saying a little documentation is good enough. Listen to these carefully. Catchy phrases are easy to understand, easy to memorize, and easy to repeat. They are very effective. These sayings (some call them proverbs) are discussed in detail in Chapter 16.

Activities, Resources, and Practices of Agile Modeling

Agile modeling involves a number of activities that need to be completed during the agile development process. This section discusses these activities, the resources, and the practices that are unique to the agile approach.

FOUR BASIC ACTIVITIES OF AGILE DEVELOPMENT Agile methods use four basic activities of development: coding, testing, listening, and designing. An agile analyst needs to identify the amount of effort that will go into each activity and balance that with the resources needed to complete the project.

Coding is designated as the one activity that it is not possible to do without. Beck (2000) states that the most valuable thing we receive from code is “learning.” The process is basically this: have a thought, code it, test it, and see whether the thought was a logical one. Code also can be used to communicate ideas that would otherwise remain fuzzy or unshaped. When others see your code, they may get a new thought (Beck, 2000).

Testing is the second basic activity of development. The agile approach views automated tests as critical. The agile approach advocates writing tests to check coding, functionality, performance, and conformance. Agile modeling relies on automated tests, and large libraries of tests exist for most programming languages. These tests need to be updated as necessary during the progress of the project.

There are both long-term and short-term reasons for testing. Testing in the short term provides you with extreme confidence in what you are building. If tests run perfectly, you can continue with renewed confidence. In the long term, testing keeps a system alive and allows you to make changes longer than would be possible if no tests were written or run.

The third basic activity of development is listening. In Chapter 4, we learned about the importance of listening during interviews. In the agile approach, listening is done in the extreme. Developers use active listening to hear their programming partner. In agile modeling, there is less reliance on formal, written communication, so listening becomes a paramount skill.

A developer also uses active listening with the customer. Developers assume that they know nothing about the business they are helping, and so they must listen carefully to businesspeople to get answers to their questions. A developer needs to come to an understanding of what effective listening is. If you don’t listen, you will not know what you should code or what you should test.

The fourth basic activity in development is designing, which is a way of creating a structure to organize all the logic in the system. Designing is evolutionary, and systems designed using the agile approach are conceptualized as evolving, as always being designed.

Good design is often simple. Design should allow flexibility as well. Designing well permits you to make extensions to the system by making changes only in one place. Effective design locates logic near the data on which it will be operating. Above all, design should be useful to all those who will need it as the development effort proceeds, including customers as well as programmers.

FOUR RESOURCE CONTROL VARIABLES OF AGILE MODELING Completing all the activities in a project on time and within all the constraints is admirable. As you have probably realized by now, however, to accomplish this, project management is crucial. Managing a project doesn't mean simply getting all the tasks and resources together. It also means that the analyst is faced with a number of trade-offs. Sometimes cost may be predetermined, and at other junctures time may be the most important factor. These resource control variables (time, cost, quality, and scope) are discussed next.

TIME You need to allow enough time to complete a project. Time, however, is split into many separate pieces. You need time to listen to the customers, time to design, time to code, and time to test.

One of our friends is an owner of a Chinese restaurant. Recently, he found himself short-staffed as one of the members of his reliable crew left the U. S. to return to Hong Kong to get married. Since he was trained as a chef, the owner placed himself in the kitchen so the food was served on time, but he stopped greeting his customers out front in the usual way. He sacrificed the listening activity to achieve another activity, but in this case he found out it was hurting his business. Customers wanted the attention the owner had always shown them and which they expected.

It is the same in systems development. You can create quality software but fail to listen. You can design a perfect system but not allow enough time to test it. Time is difficult to manage. If you find yourself running short of time, what do you do?

The agile approach challenges the notion that more time will give you the results you want. Perhaps the customer would prefer that you finish on time rather than extend the deadline to add another feature. Customers, we often find, are happy if some of the functionality is up and running on time. Our experience shows that often a customer is 80 percent satisfied with the first 20 percent of the functionality. This means that when you complete the final 80 percent of the project, the customer may be only slightly happier than he or she was after you completed the first 20 percent. The message here is be careful not to extend your deadline. The agile approach insists on finishing on time.

COST Cost is the second variable we can consider adjusting. Suppose the activities of coding, designing, testing, and listening are weighing down the project and the resources we put into time, scope, and quality are not sufficient, even with a normal amount devoted to cost, to balance the project. Essentially we might be required to contribute more resources that require money to balance the project.

The easiest way to increase spending (and hence costs) is to hire more people. This may appear to be the perfect solution. If we hire more coders, we'll finish faster. Right? Not necessarily. Picture hiring two people to repair a roof and increase that number to four. Soon the people are bumping into one another. Furthermore, they need to ask each other what still needs to be done. And if there's a lightning storm, no one will be working. Going from two to four doesn't mean it will take half the time. Consider the required increase in communication and other intangible costs when you are considering hiring more people. Remember that when new people join a team, they do not know the project or the team. They will slow down the original members because the original members must devote time to getting new members up to speed.

Overtime doesn't help much either. It increases the cost, but productivity doesn't always follow. Tired computer programmers are less effective than alert computer programmers. Tired computer programmers take a long time to complete a task, and they also make mistakes that are even more time consuming to fix.

Is there anything else we can spend our money on to facilitate finishing a project? Perhaps. A variety of tools are available to support analysts and coders. These tools are often wise investments. Analysts, for example, use graphical packages such as Microsoft Visio to communicate ideas about a project to others, and CASE tools such as Visible Analyst also help speed up projects.

Even new hardware could be a worthwhile expenditure. Laptops and smartphones improve productivity away from the office. Larger visual displays, Bluetooth-enabled keyboards, mice, and more powerful graphics cards also may increase productivity.

QUALITY The third resource control variable is quality. If ideal systems are perfect, why is so much effort placed in maintaining systems? Are we already practicing agile development by sacrificing quality in software development? In Chapter 16, we will see the importance of quality and methods (such as TQM and Six Sigma) that help to ensure that software quality is high.

The agile philosophy allows an analyst to adjust the quality resource and perhaps put less effort into maintaining quality than otherwise would be expected. Quality can be adjusted both internally

and externally. Internal quality involves testing software for factors such as functionality (Does an application do what it is supposed to do?) and conformance (Does the software meet certain conformance standards, and is it maintainable?). It usually doesn't pay to tinker with internal quality.

That leaves us with external quality, or how the customer perceives the system. The customer is interested in performance. Some of the questions a customer may ask are: Does the application act reliably (or do software bugs still exist)? Are the reports generated by the system effective? Do system-generated reports reach me on time? Does the software run effortlessly? Is the user interface easy to understand and use?

The extreme philosophy of agile development allows some of the external quality issues to be sacrificed. In order for a system to be released on time, the customer may have to contend with some software bugs. If we want to meet our deadline, the user interface may not be perfect. We can make it better in a follow-up version.

Commercial off-the-shelf software manufacturers do sacrifice quality, and it is debatable whether this is a good approach. Developers may be using extreme programming as one of their agile practices, so don't be surprised when your PC software applications (not to mention your operating system and Web browser) are updated often.

SCOPE Finally, there is scope. In the agile approach, scope is determined by listening to customers and getting them to write down their stories. Then the stories are examined to see how much can be done in a given time to satisfy the customer. Stories should be brief and easy to grasp. Stories will be described in more detail later in this chapter, but here is a brief example showing four short stories from an online air travel system. Each story is shown in bold type:

Display alternative flights.

Prepare a list of the five cheapest flights.

Offer cheaper alternatives.

Suggest to customers that they travel on other days, make weekend stays, take special promotions, or use alternate airports.

Purchase a ticket.

Allow the customer to purchase a ticket directly using a credit card (check validity).

Allow the customer to choose his or her seat.

Direct the customer to a visual display of the airplane and ask the customer to select a seat.

Ideally, the analyst would be able to determine how much time and money is needed to complete each of these stories and would be able to set the level of quality for them as well. It is obvious that this system must not sacrifice quality or credit card purchases may be invalid or customers may show up at the airport without required reservations.

Once again, agile practices allow extreme measures, so to maintain quality, manage cost, and complete a project on time, an agile analyst may want to adjust the scope of the project. This can be accomplished by agreeing with the customer that one or more of the stories can be delayed until the next version of the software. For example, maybe the functionality of allowing customers to choose their own seats can be put off for another time.

In summary, an agile analyst can control any of the four resource variables of time, cost, quality, and scope. Agility calls for extreme measures and places a great deal of importance on completing a project on time. In doing so, sacrifices must be made, and the agile analyst will find out that the available trade-offs involve difficult decisions.

FOUR CORE AGILE PRACTICES Four core practices markedly distinguish the agile approach from other approaches: short releases, the 40-hour workweek, hosting an on-site customer, and using pair programming:

1. Short releases means that the development team compresses the time between releases of the product. Rather than release a full-blown version in a year, using the short release practice, the team will tackle the most important features first, release that system or product, and then improve it later.
2. Forty-hour workweek means that agile development teams purposely endorse a cultural core practice in which the team works intensely together during a typical 40-hour workweek. As a corollary to this practice, the culture reinforces the idea that working overtime for more than a week in a row is very bad for the health of the project and the developers.

This core practice attempts to motivate team members to work intensely at the job, and then to take time off so they are relaxed and less stressed when they return. This helps team members spot problems more readily and prevents costly errors and omissions due to ineffectual performance or burnout.

3. On-site customer means that a user who is an expert in the business aspect of the systems development work is on-site during the development process. This person is integral to the process, writes user stories, communicates to team members, helps prioritize and balance the long-term business needs, and makes decisions about which feature should be tackled first.
4. Pair programming is an important core practice. It means that you work with another programmer of your own choosing. You both do coding, you both run tests. Often the senior person will take the coding lead initially, but as the junior person becomes involved, whoever has the clear vision of the goal will typically do the coding for the moment. When you ask another person to work with you, the protocol of pair programming says he or she is obligated to consent. Working with another programmer helps you clarify your thinking. Pairs change frequently, especially during the exploration stage of the development process. Pair programming saves time, cuts down on sloppy thinking, sparks creativity, and is a fun way to program.

How core agile practices interrelate with and support agile development activities, resources, and values is shown in Figure 6.3.

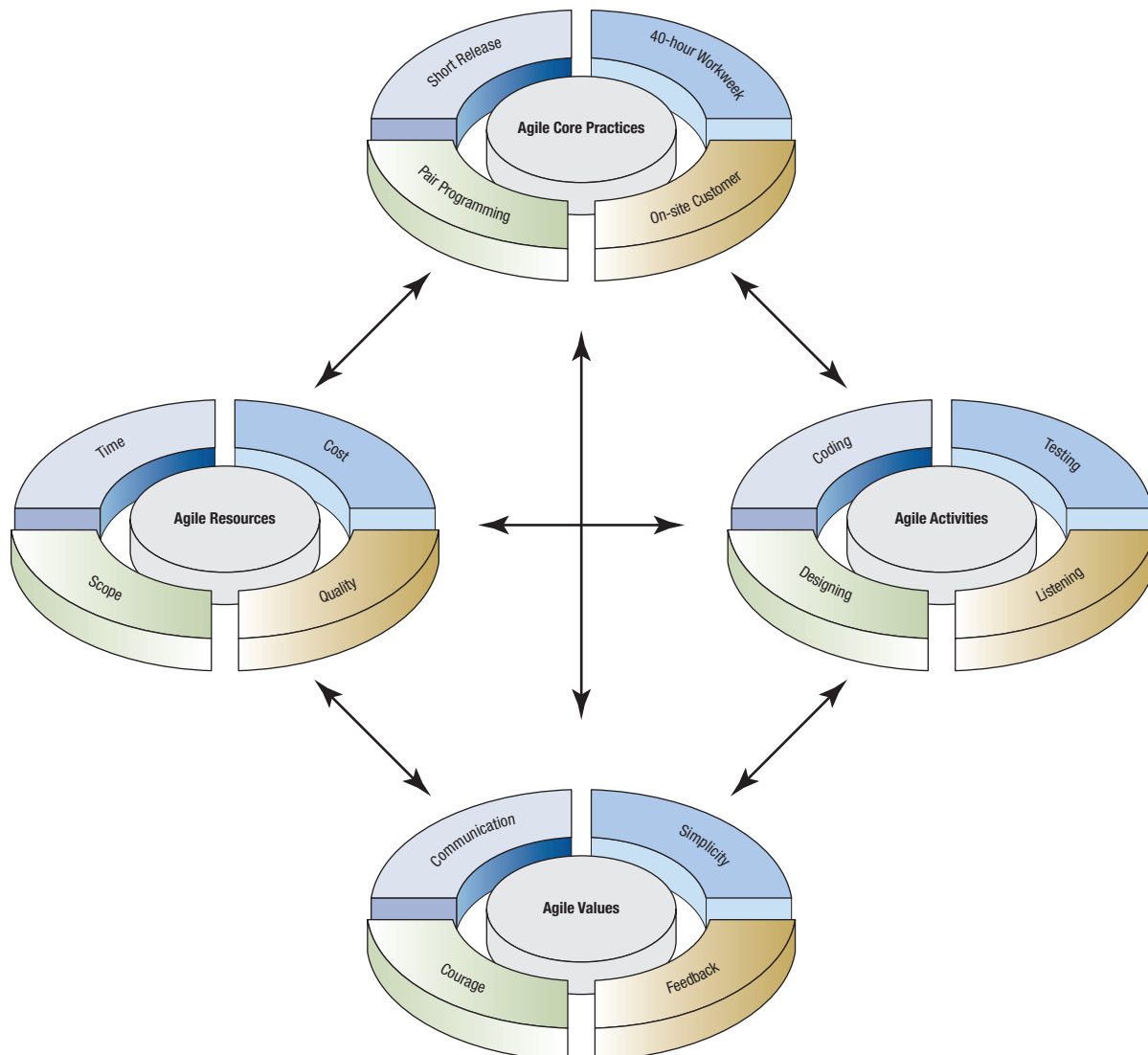


FIGURE 6.3

The core practices are interrelated with agile modeling's resources, activities, and values.

The Agile Development Process

Modeling is a keyword in agile methods. Agile modeling seizes on the opportunity to create models. These can be logical models such as drawings of systems, or mock-ups such as the prototypes described earlier in this chapter. A typical agile modeling process would go something like this:

1. Listen for user stories from the customer.
2. Draw a logical workflow model to gain an appreciation for the business decisions represented in the user story.
3. Create new user stories based on the logical model.
4. Develop some display prototypes. In doing so, show the customers what sort of interface they will have.
5. Use feedback from the prototypes and the logical workflow diagrams to develop the system until you have created a physical data model.

Agile is the other keyword in agile modeling. Agile implies maneuverability. Today's systems, especially those that are Web-based, pose twin demands: getting software released as soon as possible and continually improving the software to add new features. A systems analyst needs to have the ability and the methods to create dynamic, context-sensitive, scalable, and evolutionary applications. Agile modeling is therefore a change-embracing method.

WRITING USER STORIES Even though the title of this section is "Writing User Stories," the emphasis in the creation of user stories is on spoken interaction between developers and users, not written communication. In user stories, a developer is seeking first and foremost to identify valuable business user requirements. Users will typically engage in conversations every day with the developers about the meaning of the user stories they have written. These frequent conversations are purposeful interactions that have as their goal the prevention of misunderstandings or misinterpretations of user requirements. Therefore, user stories serve as reminders to the developers that they must hold conversations devoted to those requirements.

The following is an example of a series of stories written for an ecommerce application for an online merchant of books, CDs, movies, and other media products. The stories give a fairly complete picture of what is needed at each of the stages in the purchase process, but the stories are very short and easy to comprehend. The point here is to get all the needs and concerns of the online merchant out in the open. Although there is not enough of a story to begin coding, an agile developer might begin to see the overall picture clearly enough to begin estimating what it will take to complete the project. The stories are as follows:

Welcome the customer.

If the customer has been at this site before using this same computer, welcome the customer back to the online store.

Show specials on home page.

Show any books or other products that have recently been introduced. If the customer is identified, tailor the recommendations to that specific customer.

Search for desired product.

Include an effective search engine that will locate the specific product and similar products.

Show matching titles and availability.

Display the results of the search on a new web page.

Allow customer to ask for greater detail.

Offer the customer more product details, such as sample pages in a book, a video clip from a movie, more photos of a product, or to play a partial track from a CD.

Display reviews of the product.

Share the comments that other customers have about the product.

Place a product into a shopping cart.

Make it easy for the customer to click on a button that places the product into a shopping cart of intended purchases.



CONSULTING OPPORTUNITY 6.3

To Hatch a Fish

“Just be a little patient. I think we need to add a few more features before we turn it over to them. Otherwise, this whole prototype will sink, not swim,” says Sam Monroe, a member of your systems analysis team. All four members of the team are sitting together in a hurriedly called meeting, and they are discussing the prototype that they are developing for an information system to help managers monitor and control water temperature, number of fish released, and other factors at a large, commercial fish hatchery.

“They’ve got plenty to do already. Why, the system began with four features and we’re already up to nine. I feel like we’re swimming upstream on this one. They don’t need all that. They don’t even want it,” argues Belle Uga, a second member of the systems analysis team. “I don’t mean to carp, but just give them the basics. We’ve got enough to tackle as it is.”

“I think Monroe is more on target,” volunteers Wally Ide, a third member of the team, baiting Belle a little. “We have to show them our very best, even if it means being a few weeks later in hatching our prototype than we promised.”

“Okay,” Belle says warily, “but I want the two of you to tell the managers at the hatchery why we aren’t delivering the prototype. I don’t want to. And I’m not sure they’ll let you off the hook that easily.”

Monroe replies, “Well, I guess we could, but we probably shouldn’t make a big deal out of being later than we wanted. I don’t want to rock the boat.”

Wally chimes in, “Yeah. Why point out our mistakes to everyone? Besides, when they see the prototype, they’ll forget any complaints they had. They’ll love it.”

Belle finds a memo in her notebook from their last meeting with the hatchery managers and reads it aloud. “Agenda for meeting of September 22. ‘Prototyping—the importance of rapid development, putting together the user analyst team, getting quick feedback for modification. . . .’” Belle’s voice trails off, omitting the last few agenda items. In the wake of her comments, Monroe and Wally look unhappily at each other.

Monroe speaks first: “I guess we did try to get everyone primed for receiving a prototype quickly and to be involved from day one.” Noting your silence up until now, Monroe continues, “But still waters run deep. What do you think we should do next?” he asks you.

As the fourth member of the systems analysis team, what actions do you think should be taken? In a one- or two-paragraph email message to your teammates, answer the following questions: Should more features be added to the hatchery system prototype before giving it to the hatchery managers to experiment with? How important is the rapid development of the prototype? What are the trade-offs involved in adding more features to the prototype versus getting a more basic prototype to the client when it was promised? Complete your message with a recommendation about whether the prototype should be delayed in order to add more features or it should be delivered on time with fewer features completed.

Keep purchase history on file.

Keep details about the customer and his or her purchases in a cookie on the customer’s computer. Also keep credit card information for faster checkout.

Suggest other books that are similar.

Include photos of other books that have similar themes or were written by the same authors.

Proceed to checkout.

Confirm the identity of the customer.

Review the purchases.

Allow the customer to review the purchases.

Continue shopping.

Offer the customer an opportunity to make further purchases at the same time.

Apply shortcut methods for faster checkout.

If the identity of the customer is known and the delivery address matches, speed up the transaction by accepting the credit card on file and the remainder of the customer’s preferences, such as shipping method.

Add names and shipping addresses.

If the purchase is a gift, allow the customer to enter the name and address of the recipient along with the option to enclose a gift card.

Need or Opportunity:	Apply shortcut methods for faster checkout.					
Story:	If the identity of the customer is known and the delivery address matches, speed up the transaction by accepting the credit card on file and the rest of the customer's preferences such as shipping method.					
		Well Below	Below Average	Average	Above Average	Well Above
Activities:	Coding			✓		
	Testing			✓		
	Listening			✓		
	Designing				✓	
Resources:	Time				✓	
	Cost		✓			
	Quality				✓	
	Scope			✓		

FIGURE 6.4

User stories can be recorded on cards. The user story should be brief enough for an analyst to determine what systems features are needed.

Offer options for shipping.

Allow the customer to choose a shipping method based on cost.

Complete the transaction.

Finish the transaction. Ask for credit card confirmation if the shipping address is different from the customer's address on file.

As you can easily see, there is no shortage of stories. An agile analyst needs to choose a few stories, complete the coding, and release the product. Once this is done, more stories are selected and a new version is released until all the stories are included in the system (or the analyst and the customer agree that a particular story lacks merit or is not pressing and so need not be included).

An example of a user story as it might appear to an agile developer is shown in Figure 6.4. On paper cards (or electronically), an analyst might first identify the need or opportunity and then follow it with a brief story description. The analyst might take the opportunity to begin thinking broadly about the activities that need to be completed as well as the resources it will take to finish the project. In this example from the online merchant, the analyst indicates that the designing activity will take above-average effort, and the time and quality resources are required to rise above average. Notice that the analyst is not trying to be more precise than currently possible on this estimate, but it is still a useful exercise.

Scrum

Another agile approach is named Scrum. The word *scrum* is taken from a starting position in rugby in which the rugby teams form a huddle and fight for possession of the ball. Scrum is really about teamwork, similar to what is needed in playing a game of rugby.

Just as rugby teams will come to a game with an overall strategy, development teams begin a project with a high-level plan that can be changed on the fly as the “game” progresses. Systems development team members realize that the success of the project is most important, and their individual success is secondary. The project owner has some, but not much, influence on the detail. Rather, the tactical game is left up to the team members, just as if they were on the field. The systems team works within a strict time frame (two to four weeks for development), just as a rugby team would play in a strict time constraint of a game.

Scrum has become very popular in software development. It is an agile method that is suitable for more complex projects in which action needs to be continuous. In the development method Scrum, a limited number of features or tasks are selected for completion in a sprint,



CONSULTING OPPORTUNITY 6.4

This Prototype Is All Wet

“It can be changed. It’s not a finished product, remember,” affirms Sandy Beach, a systems analyst for RainFall, a large manufacturer of fiberglass bathtub and shower enclosures for bathrooms. Beach is anxiously reassuring Will Lather, a production scheduler for RainFall, who is poring over the first report produced for him by the prototype of the new information system.

“Well, it’s okay,” Lather says quietly. “I wouldn’t want to bother you with anything. Let’s see, . . . yes, *here* they are,” he says as he finally locates the monthly report summarizing raw materials purchased, raw materials used, and raw materials in inventory.

Lather continues looking through the multiple pages of the new computer-generated reports. “This will be fine.” Pausing at a report, he remarks, “I’ll just have Miss Fawcett copy this part for the people in Accounting.” Turning a few more pages, he says, “And the guy in Quality Assurance should really see this column of figures, although the rest of it isn’t of much interest to him. I’ll circle it and make a copy of it for him. Maybe I should phone part of this in to the warehouse too.”

As Sandy prepares to leave, Lather bundles up the pages of the reports, commenting, “The new system will be a big help. I’ll make sure everybody knows about it. Anything will be better than the ‘old monster’ anyway. I’m glad we’ve got something new.”

Sandy leaves Will Lather’s office feeling a little lost at sea. Thinking it over, he starts wondering why Accounting, Quality Assurance, and the warehouse aren’t getting what Will thinks they should. Sandy

phones a few people, and he confirms that what Lather has told him is true. They need the reports, and they’re not getting them.

Later in the week Sandy approaches Lather about rerouting the computer reports as well as changing some of the features of the system to make reports accessible online. These modifications would allow Lather to get onscreen answers regarding what-if scenarios about changes in the prices suppliers are charging or changes in the quality rating of the raw materials available from suppliers (or both), as well as allow him to see what would happen if a shipment were late.

Lather is visibly upset with Sandy’s suggestions for altering the prototype and its printed reports. “Oh, don’t do it on my account. It’s okay really. I don’t mind taking the responsibility for routing information to people. I’m always showering them with stuff anyway. Really, this is working pretty well. I would hate to have you take it away from us at this point. Let’s just leave it in place.”

Sandy is pleased that Lather seems so satisfied with the prototyped information system, but he is concerned about Lather’s unwillingness to change the prototyped reports because he has been encouraging users to think of the system as an evolving product, not a finished one.

Write a brief report to Sandy listing changes to the prototype prompted by Lather’s reactions. In a paragraph, discuss ways that Sandy can calm Lather’s fears about having the prototype “taken away.” Discuss in a paragraph some actions that can be taken *before* a prototype is tried out to prepare users for its evolutionary nature.

typically lasting two to four weeks. The result of the sprint is a potentially shippable product. Once the sprint ends, the process begins again with new priorities and features to be finished in the next sprint.

Roles Played in Scrum

There are three roles in Scrum.

1. Product owner
2. Scrum Master
3. Team member

The product owner might be viewed as a project creator because he or she expresses the vision for the product. The product owner also is responsible for initial plans for the project, releases of the product, and the final assessment.

The Scrum Master plays many roles, including serving as the team’s coach, a knowledgeable adviser, an experienced developer, and a facilitator. The Scrum Master is indeed a coach, but he or she may take on additional roles such as captain of a ship, a jungle guide who clear paths for the team, or even a guardian. A Scrum Master needs to be both knowledgeable in agile practices and have the experience to help the team. The Scrum Master is a facilitator of the team, but sometimes takes on the role of clearing obstacles for them (like the jungle guide). The culture of the group depends on the Scrum Master. He or she selects the team members, organizes and moderates Scrum meetings, and moderates conflict internally and externally.

Team members play a more significant role in Scrum. Team members:

1. Work to create and improve user stories.
2. Generate estimates.
3. Self-organize to complete the work.
4. Exhibit willingness to participate in any activity to help the project.

Teams members are chosen for their soft and technical skills. Recall that soft skills include communication and building relationships. Members may include designers, coders, a user interface expert, a tester, and a domain expert who knows about the business and the customer.

The Product Backlog

The product backlog is composed of features and other deliverables designers intend for the product based on user stories. In addition to listing features to add to an application, a product backlog also lists bugs to fix or even applications that need to be documented.

Figure 6.5 is product backlog registry. User stories are briefly described in the second column. Other columns describe who will benefit from completing each user story, what resources will be required, and how we will know when the user story is completed and accepted.

The list of user stories is reorganized so that the most important user stories appear on the top. There are many ways to prioritize user stories, but small items that can be tackled quickly are well suited for the top of the list. Although the list is not strictly in order of shortest processing time, it is encouraging to get things done. Completing many small tasks can be more encouraging than tackling one large task. User stories need to be understood by everyone on the team in order to work on the list. User stories that are not clear can be placed further down on the product backlog list.

The Sprint Cycle

Keeping a list in product backlog is one thing, but the registry doesn't accomplish anything unless the team starts working on user stories in the list. A sprint backlog is created to select the user stories that need to be completed soon. Stories are deliverables the team accomplishes. Tasks, on the other hand, are parts of the story or units of work that each team member does.

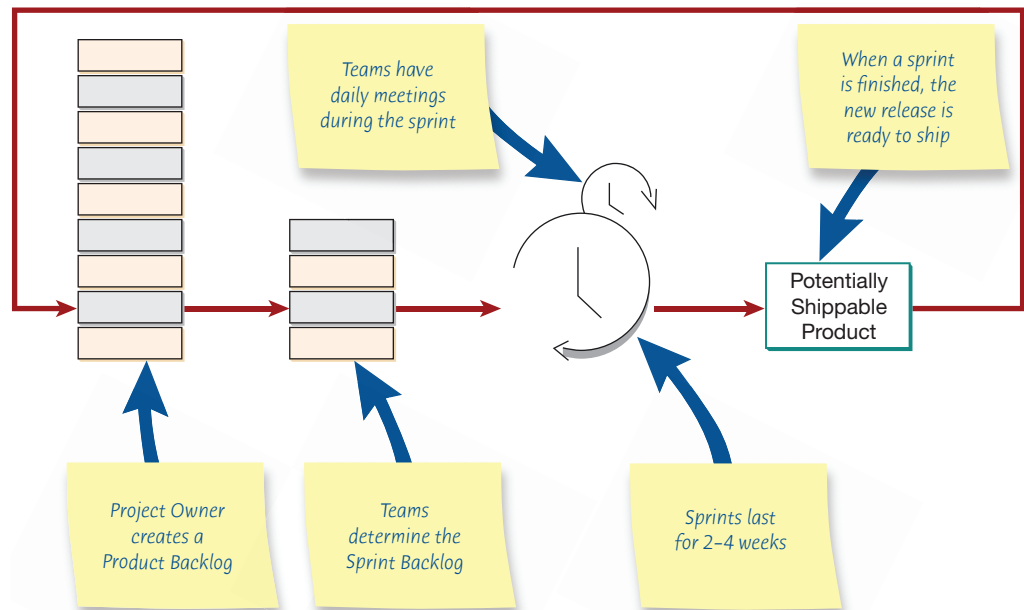
Number	Tasks based on user stories	Who will benefit	Resources required	Will be accepted when
W212	Make it easier to continue shopping once an item is placed in the shopping cart	Visitors to the website	Minor coding effort, testing	Users react positively to the prototype, coding is completed and tested
W210	Offer options for shipping earlier in the online ordering process	Visitors to the website	Dynamically calculating rates based on zip code and preferred delivery speed, coding, testing	Validity is checked, code is tested, information is approved as understandable to users
W211	Add product reviews by peers	Users who would feel more confident having these reviews available	Collaborative filtering, redesign of product pages, coding, testing	Users approve of the prototype and the collaborative system is thoroughly tested

FIGURE 6.5

A product backlog registry.

FIGURE 6.6

How teams work together to accomplish a sprint cycle for developing a new release.



The sprint cycle can vary in length, but we find that most companies set two weeks as the length of a sprint cycle. That gives the team two weeks to dispatch with a few user stories and then decide whether the product is ready to be released. That may seem impossible at first, but once a team gets into the habit of a two-week cycle, it begins to become routine and satisfying at the same time.

At the end of the two-week cycle, the team needs to determine whether the product should be released. The first question is, “Does this potential product release have value?” The product (the app or the website the team has been working on) should have value over the previous release. The second question is, “Is the product release ready to ship?” The team must have completed and tested the new features based on user stories. It must work and be free from errors.

Two-week releases have many advantages. The spirit of the team remains high because it is easy to measure accomplishments. Completing the product is more real to the team. If the team can tackle issues in a short period of time, it just takes a number of two-week releases to get the entire project completed.

Other advantages include the continual feedback the team gets from the customers. Short releases are prototypes. Customers’ reactions are very valuable.

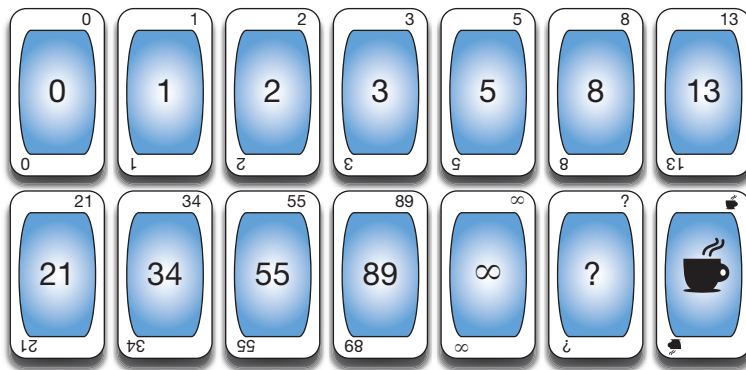
Figure 6.6 shows the process and people involved in a sprint cycle. Notice that the project owner, teams of developers, and newly developed software products all have a part to play. Sprints are short cycles of team development activity lasting only two to four weeks, with teams meeting daily during the sprint to create a new release that is ready to ship.

Other Unique Scrum Features

A few other aspects are unique to Scrum. They include a planning meeting, daily Scrum meetings, charts than monitor progress, and a sprint review.

SCRUM PLANNING MEETING There are two parts to a Scrum planning meeting. First, the product owner presents the list of features on the wish list of user stories. At this point, the team asks questions of the product owner.

The second part involves estimating the resources needed to complete all of the features. A common way to do this is to play planning poker, which is described in the next section. A card game approach to estimating makes the atmosphere of the meeting more enjoyable. Team members need to socialize, and Scrum planning meetings provide good opportunities to do so. The team members lead this part of the Scrum planning meeting. The team chooses the deliverables (user stories) that need to be accomplished. The team also divides the work into tasks. Team members are in control from this point on. They prioritize the selected user stories and make a commitment to finish a certain number of them by the end of the current sprint cycle.

**FIGURE 6.7**

A Fibonacci number-based deck of cards is used to play planning poker during a Scrum. Play the coffee card if you need a break.

SCRUM PLANNING POKER Planning poker is a way to help the team determine estimates for completing the features that arise from user stories. Planning poker is played using a deck of cards as shown in Figure 6.7. The sequence of numbers starts low and rapidly grows larger. The rationale is that there is much uncertainty in estimating larger user stories. A commonly used deck is based on Fibonacci numbers, 0, 1, 2, 3, 5, 8, 13, 21, 34, 55, and 89. (A Fibonacci number is generated by adding the two previous numbers, which guarantees that the interval between numbers becomes larger as the numbers get bigger.) In addition, decks sometimes include an infinity symbol representing a number too big to surpass, a question mark, and a coffee cup (used when members need a break from estimating). Here are the rules of the game:

1. A moderator keeps the estimation meeting running efficiently.
2. The product owner presents one of the user stories that needs to be estimated.
3. Each team member chooses a card with a number on it and lays it face down. A lower numbered card means that a project can be completed faster than higher cards estimates. The numbers may represent the number of days it may take to finish a feature, story points, or even difficulty in abstract terms.
4. Team members turn over their cards at the same time. (This avoids the problem of anchoring, or giving importance to the first estimate mentioned.)
5. The team members with the highest and lowest estimates have the opportunity to defend their ratings.
6. The entire team can now discuss the estimates. Ideally, the moderator should place a time limit on the discussion. A timer on a smartphone is a good way to limit the discussion.
7. A new hand is then played and so on. When a consensus on the time estimate for developing the features is reached, the team can move on to the next user story.

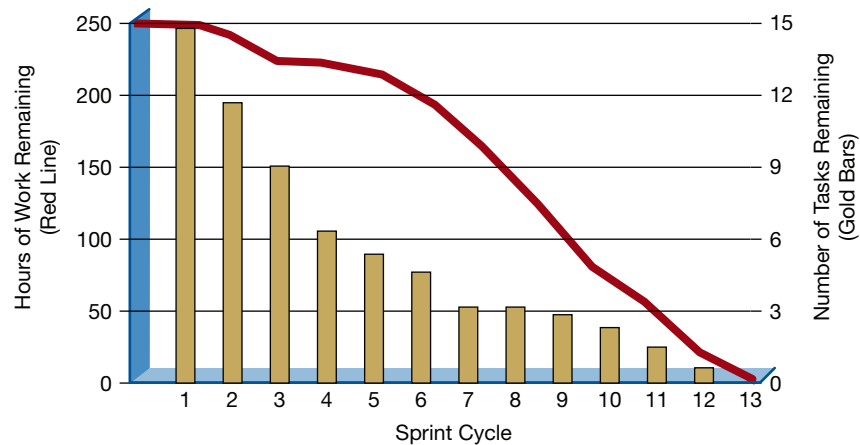
Planning poker can be a highly useful (and fun) way to develop estimates of completion times for the work designers are doing on user stories. Good estimates are difficult for inexperienced developers, so a tool like this can improve the reliability of the team's estimates without placing undue burden on newer team members.

DAILY SCRUM MEETINGS The meeting that starts off the daily Scrum is called a stand-up meeting because it lasts only a few minutes (the concept implies that team members will stand, but only for a short time). At this meeting team members tell each other what tasks they've been working on since the previous daily Scrum, what tasks they expect to complete during the current daily Scrum, and what obstacles might get in their way. Most of the time, these obstacles can't be solved during the 15-minute meeting, but it is important to identify them then.

USING BURNDOWN CHARTS One way to keep track of performance is to use a sprint burndown chart. An example of a burndown chart, which can be updated on a daily basis, is shown in Figure 6.8. The horizontal axis tracks the time that has elapsed. The vertical axis may track the number of tasks remaining or the number of hours to completion of the remaining tasks. The red line shows the hours of work remaining, and the yellow bars show the number of tasks remaining, all plotted against time (in this case in terms of a sprint cycle). Burndown charts are useful ways to graphically show the progress of an agile software development project.

FIGURE 6.8

A burndown chart shows progress on the systems project, depicting work remaining versus the time (measured in sprint cycles).



SPRINT REVIEW At the end of a sprint, the team gets together in a meeting to review the work that was done and note any tasks that were not completed. The user stories completed are prominently documented, and user stories the team committed to that were not finished are noted. The team is not making decisions for the next sprint. That will occur when the team repeats the steps starting with creating a new product backlog. This meeting is a retrospective in which the team articulates accomplishments and lessons learned. By stating what went well and what did not, the team can improve the process in anticipation of the next sprint.

Kanban

Kanban is a concept developed by Toyota to achieve a more effective and efficient way of delivering products. The word's meaning in English is "signboard," a method of directing consumers to certain areas to find products or directing employees to a location to replenish items. The concept of kanban is widely used in software development today.

Key elements of the kanban system as applied to software development are:

1. Visualize the workflow.
2. Keep work-in-process (WIP) as small as possible.
3. Reevaluate the workflow, reassigning priorities if need be.
4. Strive for continual improvement, eliminating bottlenecks and evaluating WIP limits.

It is desirable to set up a workflow summary similar to that shown in Figure 6.9, a kanban board adapted for Scrum. It was created by a company in the Netherlands and has a proprietary tool called patboard, which helps developers visually display progress on their agile projects. The cards are magnetic and washable and can be reused. They are easy to pick up and place as your project progresses, and they are small enough to depict many parts of the project but large enough to be seen from across the room where the patboard is displayed. All of this can be done using software, but the tactile nature and the easy movement of a magnetic card from one column to the next makes this process more interactive and enjoyable.

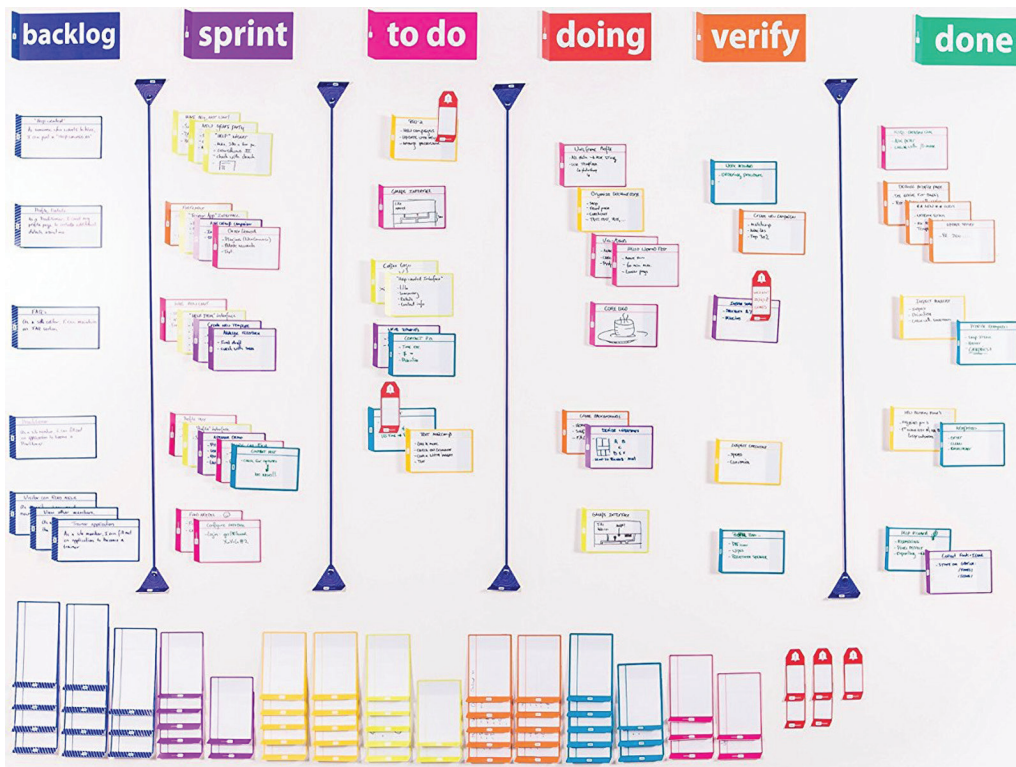
Kanban is based on a pull system. Here is an example: A new user story is introduced that states the company website must be updated. This notice means that a kanban "card" should be created. This work happened to be high priority, so it is pulled from the "backlog" column to the "sprint" column. Soon it is pulled into the "to-do" column. When the Web designers begin to work on it, they pull the task into the "doing" column. Once the new web pages are created, the task is pulled into the "verify" column so links can be tested and validated. When testing is completed, the task is pulled into the "done" column.

Note that the task was never *pushed* into another column. The concept of a pull system helps ascertain accountability and responsibility when projects require teams of people.

Cycle time is the time it takes for a task to enter the system as a "backlog" item until it reaches the "done" column. The shorter the time, the better. Throughput is the average number of items that move from the "backlog" column to the "done" column in a certain period of time or cycle. You can calculate throughput using the following equation:

$$TP = WIP/CT$$

where WIP is the work-in-progress and CT is the cycle time.

**FIGURE 6.9**

A kanban board can be created using the tools visible at www.patboard.com.

So if there are 12 user stories in progress during a four-week period, throughput is calculated as 12 divided by 4, resulting in a throughput of 3 user stories per cycle. This is, of course, a guideline. The actual number of user stories completed may vary depending on a number of factors.

The concept of kanban is applicable to all forms of agile methods. When applied to Scrum, some have used the term scrumban. They point out that scrumban is different from kanban because fixed scrum sprints are employed.

Scrum Advantages and Disadvantages

Scrum has many advantages, including:

1. Quick product development
2. Exercising a user-oriented approach
3. Encouraging teamwork
4. Being less confusing than more formal approaches
5. Flexibility
6. Being satisfying to team members
7. Rewarding smaller but meaningful accomplishments
8. Providing feedback
9. Adaptability

As one can see, Scrum has many advantages. Its disadvantages include:

1. Documenting features improperly
2. Releasing products with errors
3. Releasing products too soon for the user
4. Completing the sprint backlog under pressure
5. Working as a geographically dispersed team may be difficult
6. Working as a team when special skills are required may be challenging
7. Replacing team members who leave the team is difficult

Developers need to pay attention to minimize the effect of any disadvantages.

Scrum is a high-intensity methodology that has many advantages for developing complex and innovative packages. It is one approach that adopts the philosophy of agile modeling; however, if it becomes too intense, an organization may decide to use another agile method.

DevOps: A Cultural Shift for App Development

DevOps, which is derived from the words “Development” and “Operations,” is one way to decrease the deployment time for newly developed applications and maximize profit by rapidly addressing market opportunities and getting customer feedback in a timely manner. DevOps tries to merge these processes.

DevOps is depicted in Figure 6.10. Development and operations are working in parallel streams within a DevOps culture. Notice that development is focused on the rapid development of new applications, planning, prioritizing, designing, and coding. Operations works on testing, releasing, configuring, monitoring, and then maintaining new releases and established application software.

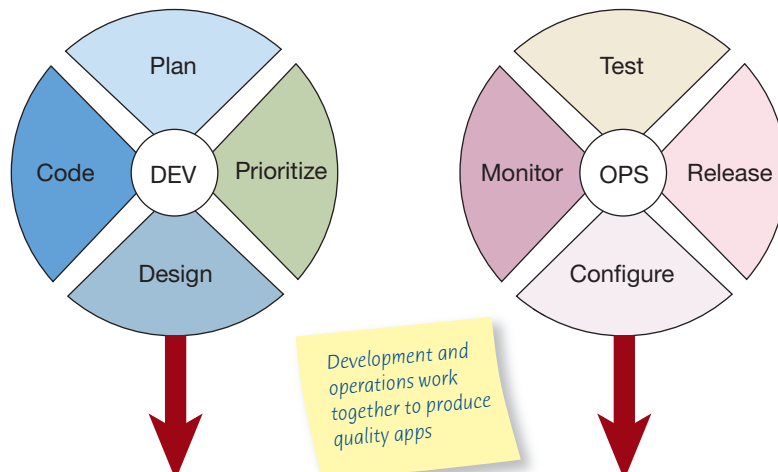
DevOps has been described as a culture rather than as a methodological approach such as SDLC or agile development because it requires a different mind-set to analyze systems problems in this way. Rather than waiting for a systems problem that arises to be solved by the development team, or arguing about where the error occurred or who is responsible (which wastes time in fixing the problem), DevOps creates two approaches in the organization that run on parallel tracks: one track supports developing rapid innovative applications, and the other supports the maintenance and operations of those processes already in place.

DevOps uses a process organization centered on value streams. Analysis of 12 to 15 processes can be drawn around each stream. Small teams of 5 to 10 people are entirely responsible for a value stream. If you’re wondering how big a DevOps team should be, you can use the Amazon two-pizza rule as a guide for team size. If the team is not small enough to be fed satisfactorily by two large pizzas, then the team is too big. The teams limit their multitasking, working on one to three projects at most. They work in small batches and small steps. They reduce the number of handoffs they do to customers who are outside the organization (external) or other departments (internal customers). They cut down or eliminate waste by avoiding partially completed work, extra processes, task switching, and tracking down defects. To detect errors early, team members “swarm” a problem so that it can be fixed immediately and not passed on at a higher cost, and so they can learn to avoid those mistakes. Quality is pushed to the team where the work is being done.

DevOps works to make their systems development efforts resilient, and they have “fire drills.” Netflix calls these drills “Chaos Monkey”: Shutting down all servers and processes to make sure the company is able to cope with worst-case scenarios. Security vulnerabilities are

FIGURE 6.10

DevOps exists in its own culture in which two parallel streams work to produce a new software release.



pushed to the DevOps teams and integrated into shared code repositories. At the conclusion of the drill, everyone is debriefed to evaluate whether the project was a success or a failure and whether it met the business's goals. If there is a security problem, a postmortem evaluation is conducted on all security breaches so that teams can learn from those problems and will not repeat them.

Comparing Agile Modeling and Structured Methods

As you have seen, agile methods are developed quickly, they reportedly work, and users are customers who are directly involved. Although it is true that projects developed using agile methods often require tweaking to work properly, agile developers state that tweaking is part of the process. The agile approach implies many short releases, with features added along the way.

Lessons Learned from Agile Modeling

Often posed as an alternative way to develop systems, the agile approach seeks to address common complaints arising about the traditional SDLC approach: being too time consuming, focusing on data rather than on humans, and being too costly. The agile approach is rapid, iterative, flexible, and participative in responding to changing human information requirements, business conditions, and environments.

Several agile development projects have been chronicled in books, articles, and on websites. Many of them were successes, some have been failures. We can learn a great deal from studying them and the agile values, principles, and core practices. Following are the six major lessons we draw from our examination of agile modeling. Figure 6.11 depicts the six lessons.

The first lesson is that short releases allow systems to evolve. Product updates are made often, and changes are incorporated quickly. In this way the system is permitted to grow and expand in ways that the customer finds useful. Through the use of short releases, the development team compresses the time between releases of their product, improving the product later as the dynamic situation demands.

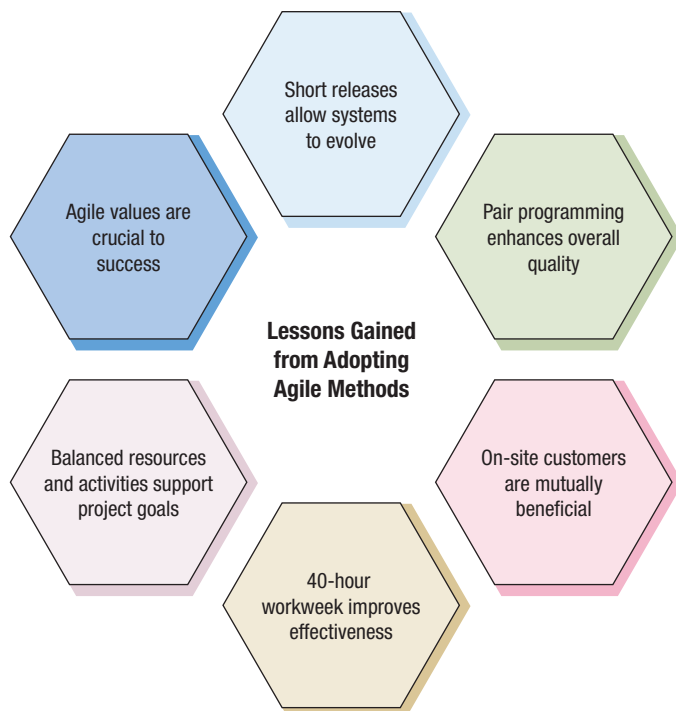


FIGURE 6.11

Six vital lessons can be drawn from the agile approach to systems.

The second lesson is that pair programming enhances overall quality. Although pair programming is controversial, it clearly fosters other positive activities necessary in systems development such as good communication, identifying with the customer, focusing on the most valuable aspects of the project first, testing all code as it is developed, and integrating the new code after it successfully passes its tests.

The third lesson is that on-site customers are mutually beneficial to the business and the agile development team. Customers serve as a ready reference and a reality check. The focus of the system design is maintained via their presence: customers become more like developers, and developers empathize more fully with customers.

The fourth lesson we take from the agile approach is that the 40-hour workweek improves effectiveness. Even the hardest-hitting developers are susceptible to errors and burnout if they work too hard for too long a period. When the development team is together, however, every moment counts. Working at a sustainable pace is much more desirable for the life of the project, the life of the system, and the life of the developer! We all know the parable of the hare and the tortoise.

The fifth lesson we draw from the agile approach is that balanced resources and activities support project goals. Managing a project doesn't mean simply getting all resources and tasks together. It also means that the analyst is faced with a number of trade-offs. Sometimes cost may be predetermined, at other junctures time may be the most important factor. The resource control variables of time, cost, quality, and scope need to be properly balanced with the activities of coding, designing, testing, and listening.

The last lesson we take from agile modeling approaches is that agile values are crucial to success. It is essential to the overall success of the project that analysts wholeheartedly embrace the values of communication, simplicity, feedback, and courage in all the work that they do. This type of personal and team commitment enables the analyst to succeed where others, who possess similar technical competencies but who lack these values, will fail. True dedication to these values is fundamental to successful development using the agile approach.

Improving Efficiency in Knowledge Work: SDLC versus Agile

Researchers Davis and Naumann (1999) developed a list of seven strategies that can improve the efficiency of knowledge work: reducing interface time and errors, reducing process learning time and dual processing losses, reducing time and effort to structure tasks and format outputs, reducing nonproductive expansion of work, reducing data and knowledge search and storage time and costs, reducing communication and coordination time and costs, and reducing losses from human information overload. Based on their study of a group of programmers, Davis and Naumann claim that the best programmers are 5 to 10 times more productive than the worst ones. They further point out this ratio is only 2:1 for workers in clerical or physical tasks. Their suggestion is that software can help improve many situations.

Adopting more software may indeed improve performance, but it is reasonable to suggest that changing an approach or a methodology may also improve performance. Consequently, we use the traditional systems development approach to examine each aspect of knowledge work productivity through the lenses of both structured and agile methodologies. Figure 6.12 lists the original seven strategies for productivity improvement and explains what methods are used to improve the efficiency of systems development for both structured and agile methodologies.

In the following sections we compare and contrast structured approaches with the agile approach. An overarching observation about the agile methodology is that it is a human-oriented approach that permits people to create nuanced solutions that are impossible to create through formal specifications of process.

REDUCING THE INTERFACE TIME AND ERRORS Systems analysts and programmers need to analyze, design, and develop systems using knowledge work tools that range from Microsoft Office to sophisticated and costly CASE tools. They also need to document as they develop systems. It is important that analysts and coders are capable of understanding the interface they use. They need to know how to classify, code, store, and write about the data they gather. Systems developers also need to quickly access an application, enter the required information, and retrieve it when it is needed again.

Strategies for Improving Efficiency in Knowledge Work	Implementation Using Structured Methodologies	Implementation Using Agile Methodologies
Reduce interface time and errors	Adopting organization standards for coding, naming, etc.; using forms	Adopting pair programming
Reduce process learning time and dual processing losses	Managing when updates are released so the user does not have to learn and use software at the same time	Ad hoc prototyping and rapid development
Reduce time and effort to structure tasks and format outputs	Using CASE tools and diagrams; using code written by other programmers	Encouraging short releases
Reduce nonproductive expansion of work	Managing project; establishing deadlines	Limiting scope in each release
Reduce data and knowledge search and storage time and costs	Using structured data gathering techniques, such as interviews, observation, sampling	Allowing for an on-site customer
Reduce communication and coordination time and costs	Separating projects into smaller tasks; establishing barriers	Timeboxing
Reduce losses from human information overload	Applying filtering techniques to shield analysts and programmers	Sticking to a 40-hour workweek

FIGURE 6.12

How Davis and Naumann's strategies for improving efficiency can be implemented using two different development approaches.

Structured approaches encourage adopting standards for everything. Rules set forth include items such as “Google Chrome will be the default on office desktops for browsing rather than Firefox.” There may be more detailed instructions to ensure clean data, such as “Always use M for Male and F for Female,” thereby ensuring that analysts do not unthinkingly choose codes of their own, such as 0 for Male and 1 for Female. These rules then become part of the data repository. Forms requiring all personnel to document their procedures are useful and are necessary in case another programmer must take over during the project.

In an agile approach, forms and procedures work well too, but another element is added. The additional practice of pair programming ensures that one programmer will check the work of another, thereby reducing the number of errors. Pair programming means that ownership of the design or software itself is shared in a partnership. Both partners (typically one a programmer, often a senior one) will say they chose a programming partner who desired to have a quality product that is error free. With two people working on the same design and code, interface time is not an issue; it is an integral part of the process. Davis and Naumann (1999) have noted that programmers are quite emotional when the topic of pair programming is broached.

REDUCING THE PROCESS LEARNING TIME AND DUAL PROCESSING LOSSES Analysts and programmers learn specific techniques and software languages required for the completion of a current project. Inefficiencies often result when some analysts and programmers already know the products used but others still need to learn them. Typically, we ask that developers learn these products as they are using them to build the system. This on-the-job training slows down the entire systems development project considerably.

A traditional, structured project requires more learning. If CASE tools are used, an analyst may need to learn the proprietary CASE tools used in the organization. The same applies to the use of a specific computer language. Documentation is also a concern.

Using an agile philosophy, the ability to launch projects without using CASE tools and detailed documentation allows analysts and programmers to spend most of their time on system development rather than on learning specific tools.



MAC APPEAL

Just as agile methodologies are an alternative to the SDLC, OmniFocus is an alternative to Microsoft Project and other Gantt chart or PERT diagram approaches.

A casual observer might think that agile methods are unstructured because systems are built without detailed specifics and documentation. A student of agile methods realizes that there is actually quite a bit of structure in the agile approach. Principles include sticking to the 40-hour work-week and coordination through pair programming. An analyst who adopts agile techniques needs a way to set goals, stay within budget, set priorities for features, and find a way to get things done.

OmniFocus is based on an alternative task management system by David Allen (2015), called *Getting Things Done*. The overriding principle is to free your mind from remembering things so that you can concentrate on completing them. An analyst using this system goes through five actions: collect, process, organize, review, and do.

A systems analyst using OmniFocus collects items from his or her Web browser, address book or calendar, and most other applications on a Mac. The analyst can categorize the data or assign it to a larger project. OmniFocus contains a planning mode so the analyst can see which task is part of a larger project and a context mode that organizes the tasks so the analyst knows all the tasks that must be done either by phone, by browsing the Web, or by using email. OmniFocus is also available as an iPhone app.

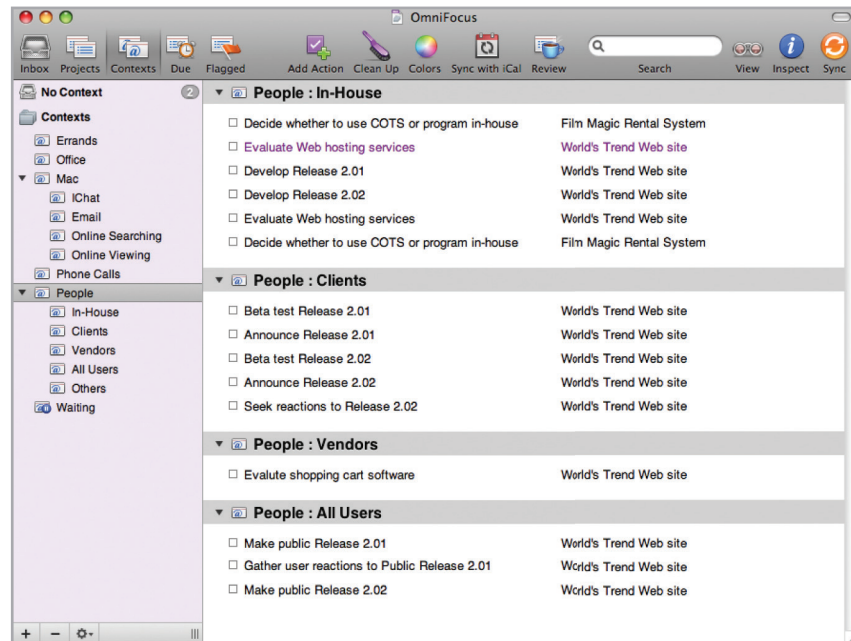


FIGURE 6.MAC

OmniFocus from The Omni Group. (Used by permission of the Omni group.)

REDUCING THE TIME AND EFFORT TO STRUCTURE TASKS AND FORMAT OUTPUTS Whenever a project is started, a developer needs to determine the boundaries. In other words, developers need to know what the deliverable will be and how they will go about organizing the project so they can complete all the necessary tasks.

A traditional approach would include using CASE tools, drawing diagrams (such as entity-relationship diagrams and data flow diagrams), using project management software (such as Microsoft Project), writing detailed job descriptions, using and reusing forms and templates, and reusing code written by other programmers.

Systems development using an agile approach addresses the need to structure tasks by scheduling short releases. The agile philosophy suggests that system developers create a series of deadlines for many releases of the system. The first releases possess fewer features, but additional features are added with each new release.

REDUCING THE NONPRODUCTIVE EXPANSION OF WORK Parkinson's law states that "work expands to fill the time available for its completion." If there are no specified deadlines, it is possible that knowledge work will continue to expand.

With traditional structured methodologies, deadlines at first seem far in the future. Analysts may use project management techniques to try to schedule the activities, but there is a built-in bias to extend earlier tasks longer than they need to be and then try to shorten tasks later on in the development. Analysts and coders are less concerned about distant deadlines than approaching ones.

Once again, the agile approach stresses short releases. Releases can be delivered at the time promised, minus some of the features originally promised. Making all deadlines imminent pushes a realistic expectation for (at least partial) completion to the fore.

REDUCING THE DATA AND KNOWLEDGE SEARCH AND STORAGE TIME AND COSTS System developers need to gather information about an organization, goals, priorities, and details about current information systems before they can proceed to develop a new system. Data-gathering methods include interviewing, administration of questionnaires, observation, and investigation by examining reports and memos.

Structured methodologies encourage structured data-gathering methods. Structured techniques would normally be used to structure interviews and design the interview process. Questionnaires would be developed in a structured way, and structured observational techniques such as Structured Observation of the Environment (STROBE) would encourage an analyst to specifically observe key elements and form conclusions based on the observations of the physical environment. A sampling plan would be determined quantitatively to provide the systems analyst with select reports and memos to examine.

Knowledge searches are less structured in an agile modeling environment. The practice of having an on-site customer greatly enhances access to information. The on-site customer is present to answer questions about the organization itself, its goals, the priorities of organizational members and customers, and whatever knowledge is necessary about existing information systems. As the project continues, the picture of customer requirements becomes clearer. This approach seems relatively painless: when the system developers want to know something, they can just ask. The downside, however, is that the on-site representative may make up information if it is unknown or unavailable or evade telling the truth for some ulterior purpose.

REDUCING COMMUNICATION AND COORDINATION TIME AND COSTS Communication between analysts and users, as well as among analysts themselves, is at the heart of developing systems. Poor communication is certainly at the root of multiple development problems. We know that communication increases when more people join a project. When two people work on a project, there is one opportunity for a one-to-one conversation; when three people are involved, there are three possibilities; when four are involved, there are six possibilities, and so on. Inexperienced team members need time to get up to speed, and they can slow down a project even though they are meant to help expedite it.

Traditional structured development encourages the separation of big tasks into smaller tasks. This allows more tightly knit groups and decreases the time spent communicating. Another approach involves setting up barriers. For example, customers may not be given access to programmers. This is a common practice in many industries. However, increased efficiency often means decreased effectiveness, and it has been noted that dividing groups and setting up barriers often introduces errors.

Agile methods, on the other hand, limit time instead of tasks. Timeboxing is used in agile methodologies to encourage completion of activities in shorter periods. Timeboxing is simply setting a time limit of one or two weeks to complete a feature or module. The agile method Scrum puts a premium on time, and the developers communicate effectively as a team. Communication is one of the four values of the agile philosophy, and communication costs tend to increase rather than decrease.

REDUCING LOSSES FROM HUMAN INFORMATION OVERLOAD We have long known that people do not react well in information-overload situations. When telephones were an emerging technology, switchboard operators manually connected calls between two parties. It was demonstrated that this system would work until an information overload occurred, at which point the entire system broke down. When too many calls came in, the overwhelmed switchboard operator would simply stop working and give up completely on connecting callers. An analogous overload situation can occur anytime to anyone, including systems analysts and coders.

A traditional approach would filter information to shield analysts and coders from customer complaints. This approach allows developers to continue working on the problem without the interference and subjectivity that would normally occur.

Using an agile philosophy, analysts and coders are expected to stick to a 40-hour workweek. Some might view this as a questionable practice. How will all the work ever get done? The agile philosophy states that quality work is usually done during a routine schedule, and it is only when overtime is added that problems of poor quality design and programming enter the scene. By sticking to a 40-hour workweek schedule, agile methodology claims that you will eventually come out ahead.

Risks Inherent in Organizational Innovation

In consultation with users, analysts must consider the risks that organizations face when adopting new methodologies. Clearly, this is part of a larger question of when is the appropriate time to upgrade human skills, adopt new organizational processes, and institute internal change.

In a larger sense, these are questions of a strategic dimension for organizational leadership. Specifically, we consider the case of the systems analysis team adopting agile methods in light of the risks to the organization and the eventual successful outcome for the systems development team and their clients. Figure 6.13 shows many of the variables that need to be considered when assessing the risk of adopting organizational innovation.

ORGANIZATIONAL CULTURE A key consideration in organizational innovation is the overall culture of the organization and how the culture of the development team fits within it. A conservative organizational culture with many stable features that does not seek to innovate may be an inappropriate or even inhospitable context for the adoption of agile methodologies by a systems development group. Analysts and other developers must use caution in introducing new techniques into this type of setting because their success is far from assured and long-standing

FIGURE 6.13

Adopting new information systems involves balancing several risks.



development team members or other organizational members may be threatened by new ways of working that depart from customary, dependable approaches with proven results.

Conversely, an organization that is dependent on innovation to retain its cutting edge in its industry might welcome agile innovations in systems development. In this instance, the culture of the organization is already permeated with the understanding of the critical nature of many of the core principles of agile development methodologies. From the strategic level downward, the company's members have internalized the need for rapid feedback, dynamic responses to changing environments in real time, dependence on the customer for guidance and participation in problem solving, and so on.

Located between these extremes are organizations that do not rely on innovation as a key strategic strength (in other words, they are not dependent on research and development of new products or services to remain afloat) but that might still wish to adopt innovative practices in small units or groups. Indeed, such small, innovative centers or kernels might eventually drive the growth or competitive advantage of this type of organization.

TIMING Organizations must ask and answer the question of when is the best time to innovate with the adoption of new systems development methodologies, taking all other projects and factors (internally and externally) into account. Organizations must consider the entire panoply of projects in which they are investing, look ahead at project deadlines, schedule the upgrading of physical plants, and absorb key industry and economic forecasts.

COST Another risk to the adoption of agile methodologies for organizations is the cost involved in education and training of systems analysts and coders in the new approach. This may involve either costly off-site seminars and courses or hiring consultants to work with current staff on-site. Furthermore, opportunity costs are involved when systems developers are necessarily diverted (albeit temporarily) from ongoing projects to learn new skills. Education in itself can be costly, but an additional burden is recognized when analysts cannot earn income during their training period.

CLIENTS' REACTIONS When clients (whether internal or external) are involved as users or initiators of information systems development efforts, reactions to the use of new methods entailed by the agile approach are also a key consideration. Some clients react with joy once the benefits of timeliness and involvement are described. Others do not want to be used for systems "experiments" with uncertain outcomes. The client-analyst relationship must be resilient enough to absorb and adapt to changes in expected behaviors. For example, the on-site presence of a client during development is a major commitment that should be thoroughly understood and agreed upon by those adopting agile methods.

MEASURING IMPACT Another consideration for organizations adopting agile methodologies is how to certify and measure that the new methods are going to facilitate successful systems development. The strengths and weaknesses of traditional structured methods used to develop information systems are well known.

Although there is ample anecdotal evidence that agile methodologies are superior for development under some conditions, their history has been brief and the methods are not yet empirically supported. Therefore, the adoption of agile methodologies carries with it the risk that systems created with them will not be successful or will not adequately interface with legacy systems. Measuring the impact of the use of agile methodologies has begun, but organizations need to be vigilant in proposing impact measurements in tandem with the adoption of new methods.

THE INDIVIDUAL RIGHTS OF PROGRAMMERS/ANALYSTS Successful systems developers (analysts and programmers) exercise creativity in their approach to their work, and they deserve the right to work in the most fruitful configuration possible. It is possible that the working requirements of new agile methods (for example, pair programming) encroach upon some basic rights of creative people to work alone or in groups as the design work dictates. There is no "one best way" to design a system, a module, an interface, a form, or a web page. In the case of systems developers, creativity, subjectivity, and the right to achieve design objectives through numerous individual paths need to be balanced against the organizational adoption of innovative approaches such as agile methodologies.

As you can see, adopting organizational innovations poses many risks to an organization as well as to individuals. We have examined the risks to an organization as a whole as well as the risks to an individual systems analyst involved in the organization's desire to innovate.



HYPERCASE EXPERIENCE 6

“Thank goodness it’s the time of year when everything is new. I love spring; it’s the most exhilarating time here at MRE. The trees are so green, with leaves in so many different shades. So many new projects to do too; so many new clients to meet. We have a new intern, Anna Mae Silver. Sometimes the newest employee is the most eager to help. Call on her if you need more answers.”

“All the newness reminds me of prototyping. Or what I know about prototyping anyway. It’s something new and fresh, a quick way to find out what’s happening.”

“I believe that we have a few prototypes already started. Sometimes our new on-site customer, Tessa Silverstone, gets involved by helping create user stories on which to build the prototypes. But the best thing about prototypes is that they can change. I don’t know anyone who’s really been satisfied with a first pass at a prototype. But it’s fun to be involved with something that’s happening fast, and something that will change.”

HYPERCASE Questions

1. Make a list of the user stories Tessa Silverstone shared as examples.
2. Locate the prototype currently proposed for use in one of MRE’s departments. Suggest a few modifications that would make this prototype even more responsive to the unit’s needs.
3. Using a word processor, construct a nonoperational prototype for a Training Department Project Reporting System. Include features brought up by the user stories you found. *Hint:* See sample screens in Chapters 11 and 12 to help you in your design.

GEMS Add Project Prototype - Mozilla Firefox

MAPLE RIDGE ENGINEERING

Add New Project

Complete the following forms to add a new project. After each form has been completed, click the Next button to move to the next form. If any data is incomplete or has been entered incorrectly, you will get an error message informing you about the nature of the problem and suggestions on how to correct the problem.

If this is a new client, click [here](#) to add the new client information.

New Project Information

Project Description		Project Initiation Date	4/27/2017
Priority Code		Category	Development
Project Size	☐ Small ☐ Medium ☐ Large	Project Status	Not Started
☐ Training ☐ Commercial	☐ Engineering ☐ Education	☐ Architecture ☐ Software Development	
Budget Allowance		Estimated Completion Date	
Client Name		Leader Number	

FIGURE 6.HC1

One of the many prototype screens found in HyperCase.

Summary

Prototyping is an information-gathering technique useful for supplementing the traditional SDLC; however, both agile methods and human-computer interaction share roots in prototyping. When systems analysts use prototyping, they are seeking user reactions, suggestions, innovations, and revision plans to make improvements to the prototype, and thereby modify system plans with a minimum of expense and disruption. The four major guidelines for developing a prototype are to (1) work in manageable modules, (2) build the prototype rapidly, (3) modify the prototype, and (4) stress the user interface.

Agile modeling is a software development approach that defines an overall plan quickly, develops and releases software quickly, and then continuously revises software to add additional features. The values of the agile approach that are shared by the customer as well as the development team are communication, simplicity, feedback, and courage. Agile activities include coding, testing, listening, and designing. Resources available include time, cost, quality, and scope.

Agile core practices distinguish agile methods, including a type of agile method called extreme programming (XP), from other systems development processes. The four core practices of the agile approach are (1) short releases, (2) 40-hour workweek, (3) on-site customer, and (4) pair programming. The agile development process includes choosing a task that is directly related to a customer-desired feature based on user stories, choosing a programming partner, selecting and writing appropriate test cases, writing the code, running the test cases, debugging it until all test cases run, implementing it with the existing design, and integrating it into what currently exists.

Scrum is an agile method suitable for more complex projects. In the development method Scrum, a limited number of features or tasks are selected for completion in a sprint, typically lasting two to four weeks. The result of the sprint is a potentially shippable product. Once the sprint ends, the process begins again with new priorities and features to be finished in the next sprint. Specific roles are played in Scrum and specific actions are taken. Members include designers, coders, a user interface expert, a tester, and someone who is a domain expert (who knows about the business and the customer). The product backlog is composed of features and other deliverables that designers intend for the product based on user stories. In addition to listing features to add to an application, a product backlog also lists bugs to fix or even applications that need to be documented. A sprint backlog is created to select the user stories that need to be completed soon. Stories are deliverables the team accomplishes. Tasks, on the other hand, are parts of the story or units of work that each team member does. The sprint cycle can vary in length, but we find that most companies set two weeks as the length of a sprint cycle. That gives the team two weeks to complete work on a few user stories and then decide whether the product is ready to be released. Then the cycle begins again.

DevOps (derived from Development and Operations) is a cultural shift in the organization. It can create two approaches in the organization that run on parallel tracks, one track supports developing rapid innovative applications, and the other supports the maintenance and operations of those processes already in place.

Systems development life cycle and agile approaches handle improving knowledge work efficiency differently. There are several inherent dangers to organizations that adopt innovative approaches, including an incompatible organizational culture, poor timing of the project, cost of training systems analysts, unfavorable client reactions to new behavioral expectations, difficulties in measuring the impact, and the possible compromise of the individual creative rights of programmers and analysts.

Keywords and Phrases

40-hour workweek	patched-up prototype
agile modeling	product backlog
agile principles	prototype
agile values	requirements planning phase
assume simplicity	Scrum methodology
burndown chart	Scrum planning poker
DevOps	selected-features prototype
embracing change	short release
extreme programming (XP)	sprint
first-of-a-series prototype	sprint backlog
implementation	sprint review
incremental change	stressing the user interface
kanban	throughput
modifying the prototype	user involvement with prototyping
nonoperational prototype	user stories
on-site customer	working in manageable modules
pair programming	

Review Questions

1. What four kinds of information is an analyst seeking through prototyping?
2. What is meant by the term *patched-up prototype*?
3. Define a prototype that is a nonworking scale model.

4. Give an example of a prototype that is a first full-scale model.
5. Define what is meant by a prototype that is a model with some, but not all, essential features.
6. Which type of prototyping would you recommend for an organization with multiple locations around the world, each with similar needs?
7. What are the two main problems identified with prototyping?
8. What are the four values that must be shared by the development team and business customers when taking an agile approach?
9. What are agile principles? Give five examples.
10. Discuss the four basic activities of development used in the agile method.
11. Name the four resource control variables used in the agile approach.
12. How do the core agile practices interrelate with activities, resources, and values?
13. What is a user story? Is it primarily written or spoken? State your choice, then defend your answer with an example.
14. What is the meaning of agile modeling, and why is it known as a change-embracing method?
15. What is Scrum?
16. Discuss the feature of Scrum used for time estimation.
17. What are four actions team members take in Scrum?
18. What should be listed in a product backlog registry?
19. List ways in which the performance of Scrum can be measured during sprint cycles.
20. What are two questions the team should ask of itself after a two-week sprint cycle has been completed?
21. What are the nine advantages of Scrum?
22. Discuss key elements of the kanban system as applied to software development.
23. What are the seven disadvantages of Scrum?
24. How does DevOps differ from methodological approaches?
25. Explain the role of “fire drills” in systems development using DevOps.
26. What two parallel tracks are used for running DevOps?
27. List the six lessons learned from agile modeling.
28. Name the seven strategies for improving efficiency in knowledge work.
29. Identify six risks in adopting organizational innovation.

Problems

1. As part of a larger systems project, Clone Bank of Clone, Colorado, wants your help in setting up a new monthly reporting form for its checking and savings account customers. The president and vice presidents are very attuned to what customers in the community are saying. They believe their customers want a checking account summary that looks like the one offered by the other three banks in town. They are unwilling, however, to commit to that form without a formal summary of customer feedback that supports their decision. Feedback will not be used to change the prototype form in any way. They want you to send a prototype of one form to one group and to send the old form to another group.
 - a. In a paragraph, discuss why it probably is not worthwhile to prototype the new form under these circumstances.
 - b. In a second paragraph, discuss a situation under which it would be advisable to prototype a new form.
2. C. N. Itall has been a systems analyst for Tun-L-Vision Corporation for many years. When you came on board as part of the systems analysis team and suggested prototyping as part of the SDLC for a current project, C. N. said, “Sure, but you can’t pay any attention to what users say. They have no idea what they want. I’ll prototype, but I’m not ‘observing’ any users.”
 - a. As tactfully as possible, so as not to upset C. N. Itall, make a list of the reasons that support the importance of observing user reactions, suggestions, and innovations in the prototyping process.
 - b. In a paragraph, describe what might happen if part of a system is prototyped and no user feedback about it is incorporated into the successive system.
3. “Every time I think I’ve captured user information requirements, they’ve already changed. It’s like trying to hit a moving target. Half the time, I don’t think they even know what they want themselves,” exclaims Flo Chart, a systems analyst for 2 Good 2 Be True, a company that surveys product use for the marketing divisions of several manufacturing companies.
 - a. In a paragraph, explain to Flo Chart how prototyping can help her to better define users’ information requirements.
 - b. In a paragraph, comment on Flo’s observation: “Half the time, I don’t think they even know what they want themselves.” Be sure to explain how prototyping can actually help users better understand and articulate their own information requirements.
 - c. In a paragraph, suggest how an interactive website featuring a prototype might address Flo’s concerns about capturing user information requirements.

4. Harold, a district manager for the multioutlet chain Sprocket's Gifts, thinks that building a prototype can mean only one thing: a nonworking scale model. He also believes that this is too cumbersome to prototype information systems and thus is reluctant to do so.
 - a. Briefly (in two or three paragraphs) compare and contrast the other three kinds of prototyping that are possible so that Harold has an understanding of what prototyping can mean.
 - b. Harold has an option of implementing one system, trying it, and then having it installed in five other Sprocket locations if it is successful. Name a type of prototyping that would fit well with this approach, and in a paragraph defend your choice.
5. "I've got the idea of the century!" proclaims Bea Kwick, a new systems analyst with your systems group. "Let's skip all this SDLC garbage and just prototype everything. Our projects will go a lot more quickly, we'll save time and money, and all the users will feel as if we're paying attention to them instead of going away for months on end and not talking to them."
 - a. List the reasons you (as a member of the same team as Bea) would give Bea to dissuade her from trying to scrap the SDLC and prototype every project.
 - b. Bea is pretty disappointed with what you have said. To encourage her, in a paragraph, explain the situations you think would lend themselves to prototyping.
6. The following remark was overheard at a meeting between managers and a systems analysis team at the Fence-Me-In fencing company: "You told us the prototype would be finished three weeks ago. We're still waiting for it!"
 - a. In a paragraph, comment on the importance of rapid delivery of a portion of a prototyped information system.
 - b. List three elements of the prototyping process that must be controlled to ensure prompt delivery of the prototype.
 - c. What are some elements of the prototyping process that are difficult to manage? List them.
7. Suppose you have just been hired for a systems development project that involves designing a mobile application that enables students to pay fees to educational institutions. As you only have a few months before product launch, discuss the strategies you would adopt to increase the efficiency of development.
8. Given the situation for Williwonk's chocolates (Problem 1 in Chapter 3), which of the four agile modeling resource variables may be adjusted?
9. Examine the collection of user stories from the online merchant shown earlier in the chapter. The online media store would now like to have you add some features to its website. Following the format shown in Figure 6.4, write a user story for these features:
 - a. Include pop-up ads.
 - b. Offer to share the details of the customer's purchases with his or her friends.
 - c. Extend offer to purchase other items.
10. Create a product backlog registry for any two user stories identified in Problem 9, listing probable beneficiaries, resources required, and indication of completion.
11. Visit an online course provider like edX and write up a dozen brief user stories to improve their website.
12. Using the stories you wrote for Problem 9, walk through the five stages of the agile development process and describe what happens at each one of the stages.

Group Projects

1. Divide your group into two smaller subgroups. Have Group 1 follow the processes specified in this chapter for creating prototypes. Using a CASE tool or a word processor, Group 1 should devise two nonworking prototype screens using the information collected in the interviews with Maverick Transport employees accomplished in the group exercise in Chapter 4. Make any assumptions necessary to create two screens for truck dispatchers. Group 2 (playing the roles of dispatchers) should react to the prototype screens and provide feedback about desired additions and deletions.
2. The members of Group 1 should revise the prototype screens based on the user comments they received. Those in Group 2 should respond with comments about how well their initial concerns were addressed with the refined prototypes.
3. As a united group, write a paragraph discussing your experiences with prototyping for ascertaining information requirements.
4. Within your united group, assign some of the roles that people take on in agile development. Make sure that one person is an on-site customer and at least two people are programmers. Assign other roles as you see fit. Simulate the systems development situation discussed in Problem 7, or have the person acting as the on-site customer choose an ecommerce business with which he or she is familiar. Assume that the customer wants to add some functionality to the website. Role play a scenario showing what each person would do if this was being approached through agile methods. Write a paragraph that discusses the constraints that each person faces in enacting his or her role.

Selected Bibliography

- Alavi, M. "An Assessment of the Prototyping Approach to Information Systems Development." *Communications of the ACM* 27, 6 (June 1984): 556–563.
- Allen, D. *Getting Things Done*. London: Penguin Publishing, 2015.
- Avison, D., and D. N. Wilson. "Controls for Effective Prototyping." *Journal of Management Systems* 3, 1 (1991): 52–62.
- Beck, K. *Extreme Programming Explained: Embrace Change*. Boston, MA: Addison-Wesley, 2000.
- Beck, K., and M. Fowler. *Planning Extreme Programming*. Boston, MA: Addison-Wesley, 2001.
- Cockburn, A. *Agile Software Development*. Boston, MA: Addison-Wesley, 2002.
- Davis, G. B., and J. D. Naumann. "Knowledge Work Productivity." In *Emerging Information Technologies: Improving Decisions, Cooperation, and Infrastructure*, 343–357, ed. K. E. Kendall. Thousand Oaks, CA: Sage, 1999.
- Davis, G. B., and M. H. Olson. *Management Information Systems: Conceptual Foundations, Structure, and Development*, 2d ed. New York, NY: McGraw-Hill, 1985.
- Fitzgerald, B., and G. Hartnett. "A Study of the Use of Agile Methods within Intel." In *Business Agility & IT Diffusion*, 187–202, ed. L. Matthiassen, J. Pries-Heje, and J. DeGross. New York, NY: Springer, 2005.
- Grehan, R. "InfoWorld Review: Tools for Web Development." *InfoWorld*, May 12, 2010. <https://www.networkworld.com/article/2209260/software/infoworld-review--tools-for-rapid-web-development.html>. Last accessed August 28, 2017.
- Gremillion, L. L., and P. Pyburn. "Breaking the Systems Development Bottleneck." *Harvard Business Review* (March–April 1983): 130–137.
- Harrison, T. S. "Techniques and Issues in Rapid Prototyping." *Journal of Systems Management* 36, 6 (June 1985): 8–13.
- Kendall, J. E., and K. E. Kendall. "Agile Methodologies and the Lone Systems Analyst: When Individual Creativity and Organizational Goals Collide in the Global IT Environment." *Journal of Individual Employment Rights* 11, 4 (2004–2005): 333–347.
- Kendall, J. E., K. E. Kendall, and S. Kong. "Improving Quality Through the Use of Agile Methods in Systems Development: People and Values in the Quest for Quality." In *Measuring Information Systems Delivery Quality*, 201–222, ed. E. W. Duggan and H. Reichgelt. Hershey, PA: Idea Group, 2006.
- Kong, S., J. E. Kendall, and K. E. Kendall. "Project Contexts and Use of Agile Software Development Methodology in Practice: A Case Study." *Journal of Academy of Business and Economics* 12, 2 (2012): 1–15.
- Loukides, M. *What Is DevOps?* Kindle ed. Sebastopol, CA: O'Reilly Media, 2012.
- McBreen, P. *Questioning Extreme Programming*. Boston, MA: Addison-Wesley, 2003.
- Naumann, J. D., and A. M. Jenkins. "Prototyping: The New Paradigm for Systems Development." *MIS Quarterly* 6, 3 (September 1982): 29–44.
- Tech World. *DevOps Handbook: A Guide to Implementing DevOps in Your Workplace*. Kindle ed. Alexandria, VA: WorldTech, 2017.
- Vii, P. *Agile Product Management: Kanban: The Kanban Guide*, 2d ed. Close Streatham Vale, UK: Pashun Consulting Ltd., 2016.